⚡ PERFORMANCE TESTING

OrangeHRM — JMeter Load Testing Suite

Apache JMeter 5.6.3 **GitHub Actions CI** **PDF + HTML Reports**

Presented by: Abhay Singh | QA Automation Engineer

Framework: OrangeHRM Playwright Automation Suite

📋 AGENDA

01 — WHAT IS PERFORMANCE TESTING?
Goals, types, and why it matters

02 — TOOL: APACHE JMETER
Why JMeter and how it integrates

03 — TEST PLAN WALKTHROUGH
Login flow — 4 HTTP steps explained

04 — LOAD CONFIGURATION
Threads, loops, ramp-up profiles

05 — KEY METRICS
Avg, P90, P99, throughput, error rate

06 — REPORTS
HTML dashboard + PDF for stakeholders

07 — CI/CD PIPELINE
Automated nightly runs + manual trigger

08 — NOTIFICATIONS
Email reports + Slack alerts

🎯 WHAT IS PERFORMANCE TESTING?

Performance testing validates how a system behaves under expected and peak load conditions — before real users encounter problems.

🏋️ LOAD TESTING

Simulate expected concurrent users — verify the app handles normal traffic without degradation.

💥 STRESS TESTING

Push beyond normal limits — find the breaking point and observe how the system recovers.

🌄 SPIKE TESTING

Sudden burst of users — validate the system's reaction to sudden traffic surges.

⏱️ ENDURANCE TESTING

Sustained load over time — detect memory leaks and resource exhaustion.

Our focus: Load testing the OrangeHRM login flow — the most critical and highest-traffic endpoint in the application.

🔧 WHY APACHE JMETER?

✅ OPEN SOURCE & FREE

No licensing costs — enterprise-grade tooling at zero cost.

✅ PROTOCOL SUPPORT

HTTP, HTTPS, REST, SOAP, JDBC, FTP — covers all OrangeHRM endpoints.

✅ PARAMETRIZABLE

Override threads, loops, ramp-up via CLI flags — zero plan edits needed.

✅ BUILT-IN HTML DASHBOARD



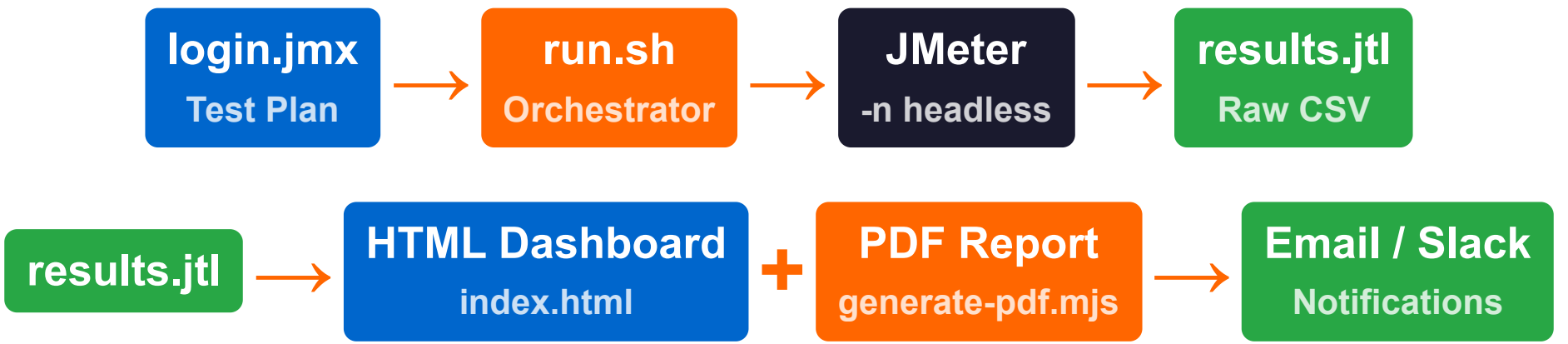
JMeter 5.6.3

- Java-based
- Headless / Non-GUI mode
- CI/CD friendly
- JTL result format

Auto-setup: JMeter is downloaded automatically by `npm run perf:setup` — no manual installation required.

Auto-generated interactive reports with graphs, percentiles, and throughput charts.

🏗️ HOW IT ALL FITS TOGETHER



📄 LOGIN.JMX

Defines the HTTP request sequence, virtual users, ramp-up, and assertions.

📊 RESULTS.JTL

Raw CSV with timestamp, response time, status, bytes, latency for every request.

📑 PDF REPORT

Generated via Playwright (Chromium) — converts the HTML dashboard to a shareable PDF.

📋 LOGIN TEST PLAN — STEP BY STEP

The **login.jmx** plan mirrors the smoke test exactly — same flow, same assertions, but executed by concurrent virtual users via HTTP.

1. **GET /web/index.php/auth/login**
   Load the login page — measures initial page load time and static asset delivery.
2. **POST /web/index.php/auth/validateCredentials**
   Submit credentials (Admin / admin123) — the most critical endpoint; measures auth server response time under load.
3. **GET /web/index.php/dashboard/index**
   Verify the dashboard loads post-login — confirms session is valid and the app serves authenticated content correctly.
4. **GET /web/index.php/auth/logout**
   Clean logout — ensures no session leak between iterations; resets state for the next loop.

Each virtual user runs **all 4 steps in sequence**, independently and concurrently with other users.

⚙️ LOAD CONFIGURATION PARAMETERS

Parameter	Flag	Default
Virtual Users	`-Jthreads=N`	1
Iterations	`-Jloops=N`	1
Ramp-up (sec)	`-Jramp_up=N`	1
Host	`-JHOST=…`	demo site

All parameters can be overridden on the command line with no changes to the .jmx file.

🔵 SMOKE PROFILE

`-Jthreads=1 -Jloops=1`

Single user, single pass — sanity check that the endpoint is alive.

🟠 BASELINE PROFILE

`-Jthreads=5 -Jloops=10 -Jramp_up=10`

5 users, 10 iterations each — establishes normal-traffic benchmarks.

🔴 STRESS PROFILE

`-Jthreads=50 -Jloops=20 -Jramp_up=60`

50 concurrent users — finds performance limits and breaking points.

KEY METRICS — WHAT WE MEASURE

Metric	What It Means	Target (Green)	Warning	Critical
Avg Response Time	Mean time to receive a full response	**< 2,000 ms**	**2–4 s**	**> 4 s**
P90 Response Time	90% of requests finish within this time	**< 3,000 ms**	**3–5 s**	**> 5 s**
P99 Response Time	Worst-case — only 1% of users are slower	**< 5,000 ms**	**5–8 s**	**> 8 s**
Throughput	Requests handled per second	**Higher = Better**	—	Drops under load
Error Rate	% of HTTP 4xx/5xx responses	**0%**	**< 1%**	**> 1%**
Latency	Time until first byte received from server	**< 500 ms**	**500ms–1s**	**> 1 s**

Why P90 over Average? Averages are skewed by fast responses. P90 tells you what 90% of real users actually experience — a much more honest measure.

REPORTS — WHAT GETS GENERATED

HTML DASHBOARD

`jmeter/reports/login/index.html`

Interactive charts powered by JMeter's built-in dashboard generator:

- Response times over time
- Throughput per second
- Error rate timeline
- Percentile distribution (P50/P90/P99)
- Transactions per second

PDF REPORT

`login-performance-report.pdf`

Auto-generated from the HTML dashboard using Playwright's Chromium PDF engine:

- All charts captured as high-res images
- Summary table (avg, max, P90, error%)
- Ready to email to stakeholders
- Attached to GitHub Actions run artifacts
- Retained for 30 days per run

STATISTICS.JSON

LOGIN-SUMMARY.JTL

Machine-readable summary — consumed by CI to post metrics directly into the GitHub Actions job summary panel.

Raw CSV results file — every individual request row. Can be loaded into any BI tool for custom analysis.

🚀 HOW TO RUN — STEP BY STEP

❶ One-time setup — download JMeter

```
npm run perf:setup  # Downloads Apache JMeter 5.6.3 (~60 MB)
```

❷ Run the login performance test (default: 1 user, 1 loop)

```
npm run perf:login
```

❸ Run with custom load (e.g. 10 users, 5 loops, 30s ramp-up)

```
bash jmeter/scripts/run.sh login -Jthreads=10 -Jloops=5 -Jramp_up=30
```

❹ Open the HTML dashboard in your browser

```
npm run perf:report:login
```

❺ Regenerate PDF from existing results

```
npm run perf:pdf
```

🔄 CI/CD — GITHUB ACTIONS PIPELINE

TRIGGERS

🌙 NIGHTLY SCHEDULE

Runs automatically every day at **01:00 UTC** — continuous performance baseline monitoring.

🖱️ MANUAL TRIGGER

GitHub Actions UI → *Run workflow* — pick threads, loops, ramp-up from dropdowns.

🔗 WORKFLOW CALL

WHAT CI DOES

1. Sets up Java 17, Node 20, and downloads JMeter (cached)
2. Installs Playwright Chromium for PDF generation
3. Runs the JMeter test plan with configured parameters
4. Uploads PDF + HTML + JTL as 30-day artifacts
5. Posts avg/max response time + error rate to job summary
6. Sends email report (if MAIL_* secrets set)
7. Posts Slack notification with pass/fail status + link

🔔 NOTIFICATIONS & INTEGRATIONS

📧 EMAIL REPORT

After each CI run, a PDF report is sent via SMTP to the configured recipient. The email includes:

- Run summary (plan, users, loops)
- Avg + Max response time
- Error rate
- Link to GitHub Actions run
- PDF attached

💬 SLACK NOTIFICATION

A Slack message is posted to the configured channel with:

- Pass ✅ or Fail ❌ status
- Plan name + virtual user count
- Branch name
- Direct link to the GitHub Actions run
- Response time metrics

Configuration: Both integrations are optional and activated by setting GitHub repository secrets: `MAIL_HOST`, `MAIL_USER`, `MAIL_PASS`, `MAIL_TO` for email | `SLACK_WEBHOOK_URL` for Slack. No code changes required — just add the secrets in GitHub → Settings → Secrets.

✅ SUMMARY

JMeter 5.6.3

INDUSTRY-STANDARD TOOL

Auto-installed, no manual setup required

4 Steps

LOGIN FLOW TEST PLAN

Mirrors smoke test — end-to-end login, dashboard, logout

3 Profiles

SMOKE → BASELINE → STRESS

Fully configurable via CLI flags — no .jmx edits

PDF + HTML

AUTO-GENERATED REPORTS

Ready for stakeholders immediately after each run

Nightly CI

GITHUB ACTIONS

01:00 UTC automated baseline + manual trigger

Email + Slack

INSTANT NOTIFICATIONS

Team informed automatically — no manual checks



QUESTIONS & DISCUSSION

📧 qaabhayraj@gmail.com

🔗 github.com/qaabhayraj-cpu/DemoPlaywriteAI-AutomationProject

📊 Run: `npm run perf:login`